Kafka Practice - Task Answers

> Repository: GitHub - EPAM TechOrda Big Data

Task 1: Raising Kafka Using Docker

Commands used:

```
docker network create bigdata_network
docker create --name kafka --network bigdata_network -p 2181:2181 -p 9092:9092 -p 8083:8083 -e ADVERTISED_HOST=192.168.16.1 ofrir119
docker start kafka
docker exec -it kafka bash
```

- **Kafka location:** `/opt/kafka_2.12-2.4.0`
- **ZooKeeper port:** `2181`

Task 2: Creating Kafka Topics

Commands used:

```
./kafka-topics.sh --zookeeper localhost:2181 --list
./kafka-topics.sh --zookeeper localhost:2181 --create --topic kafka-tst-01 --partitions 1 --replication-factor 1
./kafka-topics.sh --zookeeper localhost:2181 --describe --topic kafka-tst-01
```

Result: Topic `kafka-tst-01` created successfully with 1 partition and replication factor 1.

Task 4: Writing and Reading Kafka Topics

Producer command:

```
./kafka-console-producer.sh --broker-list localhost:9092 --topic kafka-tst-01
```

Consumer command:

```
./kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic kafka-tst-01
```

Consumer from beginning:

```
./kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic kafka-tst-01 --from-beginning
```

Delete topic:

```
./kafka-topics.sh --zookeeper localhost:2181 --delete --topic kafka-tst-01
```

Key answers:

- Multiple producers CAN write to the same topic
- Multiple consumers CAN read from the same topic
- Without consumer group: all consumers receive ALL messages
- With consumer group: round-robin distribution between consumers

---

Task 5: Kafka Connect - File Source

Directories created: `/kafka/confFiles`, `/kafka/srcFiles`, `/kafka/sinkFiles`

Source config file: `/kafka/confFiles/connect-file-source.properties`

```
name=kafka-file-source-task
connector.class=org.apache.kafka.connect.file.FileStreamSourceConnector
tasks.max=1
file=/kafka/srcFiles/sourceFile.log
topic=kafka-file-topic
```

Run command:

```
./connect-standalone.sh /opt/kafka_2.12-2.4.0/config/connect-standalone.properties /kafka/confFiles/connect-file-source.properties
```

Result: Records written to source file were automatically published to Kafka topic. New records consumed in real-time.

Task 6: Kafka Connect - File Sink

Source config: `/kafka/confFiles/connect-file-source.properties`

```
name=kafka-file-source-task-2
connector.class=org.apache.kafka.connect.file.FileStreamSourceConnector
tasks.max=1
file=/kafka/srcFiles/newSourceFile.log
topic=kafka-file-sink-topic
```

Sink config: `/kafka/confFiles/connect-file-sink.properties`

```
name=kafka-file-sink-task
connector.class=org.apache.kafka.connect.file.FileStreamSinkConnector
tasks.max=1
file=/kafka/sinkFiles/targetFile.log
topics=kafka-file-sink-topic
```

Run command:

```
./connect-standalone.sh /opt/kafka_2.12-2.4.0/config/connect-standalone.properties /kafka/confFiles/connect-file-source.properties /
```

Result: Data flows from source file → Kafka topic → destination file in real-time.

Task 7: Kafka Administration

List consumer groups:

```
./kafka-consumer-groups.sh --bootstrap-server localhost:9092 --list
```

Describe consumer group:

```
./kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --group connect-kafka-file-sink-task
```

Consumer with specific group:

```
./kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic kafka-file-sink-topic --group test-admin-group --from-beginnir
```

Kafka Connect REST API:

```
curl localhost:8083/connectors | python -m json.tool
curl localhost:8083/connectors/kafka-file-sink-task/tasks | python -m json.tool
curl localhost:8083/connectors/kafka-file-sink-task/status | python -m json.tool
curl localhost:8083/connectors/kafka-file-sink-task/config | python -m json.tool
```

Reset offsets to earliest:

```
./kafka-consumer-groups.sh --bootstrap-server localhost:9092 --group connect-kafka-file-sink-task --topic kafka-file-sink-topic --re
```

Key answers:

- Each consumer group maintains independent offsets
- Offsets cannot be reset while consumer is active
- After offset reset: all records reprocessed
- Without offset reset: consumption resumes from last committed offset

---

Task 8: Kafka Connect - JDBC Source (MySQL)

Start MySQL:

```
docker start mysql
docker exec -it mysql /bin/bash
mysql -uroot -proot
```

Database setup:

```
CREATE DATABASE srcdb;
CREATE TABLE src_events (event_id INT PRIMARY KEY, event_timestamp TIMESTAMP NOT NULL);
```

JDBC config: `/kafka/confFiles/connect-jdbc-source.properties`

```
name=Kafka-jdbc-source-task-1
connector.class=io.confluent.connect.jdbc.JdbcSourceConnector
connection.url=jdbc:mysql://host.docker.internal:3306/srcdb?user=root&password=root&allowPublicKeyRetrieval=true
table.whitelist=src_events
tasks.max=1
poll.interval.ms=2000
mode=incrementing
incrementing.column.name=event_id
topic.prefix=mysql-src-
```

Run command:

```
./connect-standalone.sh /opt/kafka_2.12-2.4.0/config/connect-standalone.properties /kafka/confFiles/connect-jdbc-source.properties
```

Topic created: `mysql-src-src_events`

Delete topic:

```
./kafka-topics.sh --zookeeper localhost:2181 --delete --topic mysql-src-src_events
```

Result: Data ingested from MySQL to Kafka. Incrementing mode detects new rows automatically based on `event_id` column.